

[API Reference](@id api-reference)

Complete API documentation for AlignedAllocs.jl.

Table of Contents

```
Pages = ["api.md"]
Depth = 3
```

Exported Functions

Vector Allocations

memalign_vec

```
memalign_vec
```

```
memalign_vec(::Type{T}, nitems::Integer; align::Integer=CACHE_LINE_SIZE) where T
```

Allocate aligned storage for `nitems` elements of type `T` and return a `Vector{T}` whose data pointer is aligned to `align` bytes.

Arguments:

- `T::Type`: Element type (must be a bits type)
- `nitems::Integer`: Number of elements to allocate
- `align::Integer=CACHE_LINE_SIZE`: Alignment in bytes (must be power of 2 ≥ 16)

Returns:

- `Vector{T}`: Aligned vector with `nitems` elements

Throws:

- `ArgumentError`: If `T` is not a bits type, `nitems  $\leq 0$` , or `align` is invalid
- `OutOfMemoryError`: If allocation fails due to insufficient memory
- `OverflowError`: If `nitems * sizeof(T)` overflows

Examples:

```
# Allocate 1000 Float64 elements with default (cache line) alignment
v = memalign_vec(Float64, 1000)
```

```
# Allocate with custom 256-byte alignment
v = memalign_vec(Float64, 1000; align=256)

# Works with any bits type
v_int = memalign_vec(Int64, 500)
v_complex = memalign_vec(ComplexF64, 100)
```

See also: `memalign`, `memalign_clear_vec`, `alignment`

`memalign`

```
memalign(::Type{T}, nitems::Integer; align::Integer=CACHE_LINE_SIZE) where T
```

Alias for `memalign_vec`. Provides backward compatibility.

Examples:

```
v = memalign(Float64, 1000)
# Equivalent to:
v = memalign_vec(Float64, 1000)
```

`memalign_clear_vec`

```
memalign_clear_vec
```

```
memalign_clear_vec(::Type{T}, nitems::Integer; align::Integer=CACHE_LINE_SIZE)
where T
```

Allocate aligned storage for `nitems` elements of type `T`, zero-initialize it, and return a `Vector{T}`.

Arguments:

- `T::Type`: Element type (must be a bits type)
- `nitems::Integer`: Number of elements to allocate
- `align::Integer=CACHE_LINE_SIZE`: Alignment in bytes (must be power of 2 ≥ 16)

Returns:

- `Vector{T}`: Zero-initialized aligned vector with `nitems` elements

Throws:

- Same as `memalign_vec`

Examples:

```
# All elements are guaranteed to be zero
v = memalign_clear_vec{Int64, 1000}
@assert all(iszero, v)

# With custom alignment
v = memalign_clear_vec{Float64, 500; align=128}
```

Performance Note:

Zero-initialization has minimal overhead (uses optimized `memset`), but skip it if you'll immediately overwrite all values:

```
# If you'll initialize immediately, use memalign_vec
v = memalign_vec{Float64, 1000}
fill!(v, 3.14) # Immediate initialization

# If you need zero initialization, use memalign_clear_vec
v = memalign_clear_vec{Float64, 1000}
```

See also: `memalign_vec`, `memalign_clear`

`memalign_clear`

```
memalign_clear{::Type{T}, nitems::Integer; align::Integer=CACHE_LINE_SIZE} where T
```

Alias for `memalign_clear_vec`. Provides backward compatibility.

Examples:

```
v = memalign_clear{Float64, 1000}
# Equivalent to:
v = memalign_clear_vec{Float64, 1000}
```

Fixed-Size Array Allocations

These functions require `FixedSizeArrays.jl` to be available.

`memalign_fix`

```
memalign_fix
```

```
memalign_fix(::Type{T}, dims::Tuple{Vararg{Integer,N}};
align::Integer=CACHE_LINE_SIZE) where {T,N}
memalign_fix(::Type{T}, n::Integer; align::Integer=CACHE_LINE_SIZE) where {T}
```

Allocate aligned storage for a `FixedSizeArray` of element type `T` with shape `dims`. The returned array uses `memalign_vec` and preserves the alignment guarantee for the underlying dense storage.

Arguments:

- `T::Type`: Element type (must be a bits type)
- `dims::Tuple` or `n::Integer`: Array dimensions
- `align::Integer=CACHE_LINE_SIZE`: Alignment in bytes

Returns:

- `FixedSizeArray{T,N}`: Fixed-size array with aligned storage

Throws:

- `ArgumentError`: If dimensions are invalid (empty, zero, or negative)
- `OverflowError`: If dimension product overflows `Int`
- Same errors as `memalign_vec`

Examples:

```
using FixedSizeArrays

# 1D fixed array
v = memalign_fix(Float64, 100)

# 2D fixed array (matrix)
m = memalign_fix(Float64, (10, 20))
@assert size(m) == (10, 20)

# 3D array
arr = memalign_fix(Int32, (5, 10, 15))

# With custom alignment
m = memalign_fix(Float64, (8, 8); align=256)
```

See also: `memalign_clear_fix`, `memalign_seq`

`memalign_clear_fix`

```
memalign_clear_fix
```

```
memalign_clear_fix(::Type{T}, dims::Tuple{Vararg{Integer,N}};
align::Integer=CACHE_LINE_SIZE) where {T,N}
memalign_clear_fix(::Type{T}, n::Integer; align::Integer=CACHE_LINE_SIZE) where
{T}
```

Allocate aligned storage for a `FixedSizeArray` of element type `T` with shape `dims` and initialize all entries to zero.

Arguments:

- Same as `memalign_fix`

Returns:

- `FixedSizeArray{T,N}`: Zero-initialized fixed-size array with aligned storage

Examples:

```
# Zero-initialized 1D array
v = memalign_clear_fix{Int64, 50}
@assert all(iszero, v)

# Zero-initialized 2D array
m = memalign_clear_fix{Float64, (4, 4)}
@assert all(iszero, m)
```

See also: `memalign_fix`, `memalign_clear_vec`

`memalign_seq`

```
memalign_seq
```

```
memalign_seq(::Type{T}, nvectors, nitems_per_vector; align=CACHE_LINE_SIZE) where
T
```

Allocate `nvectors` fixed-size vectors of type `T`, each with `nitems_per_vector` elements. Each vector is aligned to `align` bytes, and the underlying storage is zero-initialized.

Arguments:

- `T::Type`: Element type (must be a bits type)
- `nvectors::Integer`: Number of vectors to create
- `nitems_per_vector::Integer`: Number of elements per vector
- `align::Integer=CACHE_LINE_SIZE`: Alignment in bytes

Returns:

- `NTuple{nvectors, FixedSizeVector{T}}`: Tuple of aligned fixed-size vectors

Memory Layout:

Each vector starts at an aligned boundary with padding between them:

```
[Vector 1][pad...][Vector 2][pad...][Vector 3][pad...]
^           ^           ^
align      2*align     3*align
```

Examples:

```
# Create 4 vectors, each with 10 elements
vecs = memalign_seq(Float64, 4, 10)
@assert length(vecs) == 4

for v in vecs
    @assert length(v) == 10
    @assert all(iszero, v)
end

# With custom alignment (useful for multi-threading)
vecs = memalign_seq(Int32, Threads.nthreads(), 1000; align=256)
```

Use Cases:

1. **Multi-threaded computation** - Each thread gets its own aligned vector:

```
vecs = memalign_seq(Float64, Threads.nthreads(), 10000)
Threads.@threads for i in 1:Threads.nthreads()
    process!(vecs[i]) # No false sharing
end
```

2. **SIMD batching** - Process multiple datasets with aligned access:

```
datasets = memalign_seq(Float32, 8, 256)
# Process in parallel with SIMD
```

See also: `memalign_fix`, `memalign_clear_fix`, `alignment`

Alignment Detection

`alignment`

```
alignment
```

```
alignment(xs::AbstractArray) -> Int
alignment(xs::NTuple{N,T}) -> Int
```

Return the largest power-of-two alignment that divides the data pointer of `xs`.

Arguments:

- `xs::AbstractArray`: Array to check alignment of
- `xs::NTuple`: Tuple of arrays (returns minimum alignment)

Returns:

- `Int`: Maximum power-of-2 alignment in bytes, or 0 for empty arrays

Examples:

```
# Check aligned array
v = memalign_vec(Float64, 100; align=256)
a = alignment(v)
@assert a >= 256
@assert ispow2(a)

# Check regular array
v = Vector{Float64}(undef, 100)
a = alignment(v) # Typically 16, but not guaranteed

# Check multiple arrays
v1 = memalign_vec(Float64, 100; align=64)
v2 = memalign_vec(Float64, 100; align=128)
min_align = alignment((v1, v2)) # Returns 64 (minimum)
```

Algorithm:

Uses bit manipulation to extract the largest power of 2 dividing the address:

```
addr = UInt(pointer(xs))
return addr == 0 ? 0 : Int(addr & -addr)
```

Empty Arrays:

Empty arrays (length 0) don't own storage, so `alignment` returns 0:

```
v = Int64[]  
@assert alignment(v) == 0
```

See also: `memalign_vec`, `memalign_clear_vec`

Constants

CACHE_LINE_SIZE

```
const CACHE_LINE_SIZE::Int
```

The detected cache line size of the CPU in bytes. Automatically detected at package precompilation time using platform-specific system calls.

Typical Values:

- Most modern CPUs: 64 bytes
- Some older CPUs: 32 bytes
- Some high-performance CPUs: 128 bytes

Fallback: If detection fails, defaults to 64 bytes (`FallbackCacheLineSize`).

Platform-Specific Detection:

- **Linux/Unix:** Uses `sysconf(_SC_LEVEL1_DCACHE_LINESIZE)`
- **macOS:** Uses `sysctlbyname("hw.cachelinesize")`
- **Windows:** Uses `GetLogicalProcessorInformationEx`

Examples:

```
using AlignedAllocs  
  
println("Cache line size: ", AlignedAllocs.CACHE_LINE_SIZE, " bytes")  
  
# Use default alignment (cache line)  
v = memalign_vec(Float64, 1000) # Uses CACHE_LINE_SIZE
```

FallbackCacheLineSize

```
const FallbackCacheLineSize = 64
```

Default cache line size (64 bytes) used when automatic detection fails.

Internal Functions

These functions are not exported but may be useful for advanced users.

check_args

```
check_args(::Type{T}, nitems::Integer, align::Integer) where T -> Nothing
```

Validate allocation arguments. Throws `ArgumentError` if:

- `T` is not a bits type
 - `nitems ≤ 0`
 - `align` is not a valid alignment (power of 2, ≥ 16)
-

is_alignment_valid

```
is_alignment_valid(align::Integer) -> Bool
```

Check if `align` is a valid alignment value:

- Must be ≥ 16
- Must be a power of 2

Implementation:

```
align >= 16 && ((align - 1) & align) == 0
```

The bit manipulation `(align - 1) & align == 0` tests for power of 2.

confirm_alignment

```
confirm_alignment(ptr::Ptr, align::Integer) -> Nothing
```

Verify that `ptr` is actually aligned to `align` bytes. Throws `ErrorException` if not aligned (and frees the memory).

zeromem

```
zeromem(ptr::Ptr{UInt8}, nbytes::Int) -> Nothing
```

Zero-initialize `nbytes` bytes starting at `ptr`. Uses `Base.memset` for efficiency.

_nbytes

```
_nbytes(::Type{T}, n::Integer) where T
```

Compute the number of bytes needed to store `n` elements of type `T`. Uses `Base.checked_mul` to detect overflow.

Error Types

Thrown Exceptions

ArgumentError

Thrown when arguments are invalid:

Causes:

- Non-bits type: `memalign_vec(String, 10)`
- Invalid size: `memalign_vec(Int64, 0)` or `memalign_vec(Int64, -5)`
- Invalid alignment: `memalign_vec(Int64, 10; align=10)` (not power of 2)
- Invalid alignment: `memalign_vec(Int64, 10; align=8)` (too small)
- Empty dimensions: `memalign_fix(Float64, ())`

Example:

```
try
    v = memalign_vec(String, 100)
catch e
    @assert e isa ArgumentError
    @assert occursin("bitstype", e.msg)
end
```

OutOfMemoryError

Thrown when system cannot allocate requested memory.

Causes:

- Insufficient available memory
- Memory fragmentation
- Per-process memory limits

Example:

```
try
  # Try to allocate huge amount
  v = memalign_vec(Float64, typemax(Int) ÷ 2)
catch e
  @assert e isa OutOfMemoryError
end
```

OverflowError

Thrown when size calculation overflows `Int`.

Causes:

- `nitems * sizeof(T)` exceeds `typemax(Int)`
- Fixed-size array dimension product exceeds `typemax(Int)`

Example:

```
try
  # Dimension product overflows
  m = memalign_fix(Float64, (typemax(Int) ÷ 2, typemax(Int) ÷ 2))
catch e
  @assert e isa OverflowError
end
```

ErrorException

Thrown for unexpected allocation failures, such as when the allocated pointer is not properly aligned (indicates a bug in the system allocator).

Type Requirements

Bits Types

Only **bits types** (types without pointers) can be allocated:

Valid Types:

```
# Primitive numeric types
Int8, Int16, Int32, Int64, Int128
UInt8, UInt16, UInt32, UInt64, UInt128
Float16, Float32, Float64
Bool

# Complex numbers
ComplexF32, ComplexF64

# Custom structs (if all fields are bits types)
struct Point3D
    x::Float64
    y::Float64
    z::Float64
end
memalign_vec(Point3D, 100) # ☒ OK
```

Invalid Types:

```
# Types with pointers
String # ✗
Vector{Int} # ✗
Array{Float64} # ✗
Any # ✗

# Mutable structs with pointers
mutable struct Node
    value::Int
    next::Union{Node, Nothing}
end
memalign_vec(Node, 10) # ✗ Error
```

Checking if a Type is Valid:

```
@assert isbitstype(Float64) # ☒ true
@assert !isbitstype(String) # ☒ false
@assert isbitstype(Point3D) # ☒ true (if defined as above)
```

Platform-Specific Behavior

POSIX Systems (Linux, macOS, BSD)

Allocation Function: `posix_memalign`

Characteristics:

- Memory freed automatically by Julia's GC
 - Alignment must be multiple of `sizeof(Ptr)`
 - Error codes: `EINVAL` (invalid alignment), `ENOMEM` (out of memory)
-

Windows

Allocation Function: `_aligned_malloc` and `_aligned_free`

Characteristics:

- Custom finalizer required (uses `_aligned_free`)
 - No restriction on alignment beyond power-of-2
 - Returns `NULL` on failure
-

Performance Tips

1. Choose Appropriate Alignment

```
# For SSE (128-bit): 16-byte alignment (default for regular arrays)
v = memalign_vec(Float64, n; align=16)

# For AVX (256-bit): 32-byte alignment
v = memalign_vec(Float64, n; align=32)

# For AVX-512 (512-bit): 64-byte alignment
v = memalign_vec(Float64, n; align=64)

# For cache line: use default
v = memalign_vec(Float64, n) # Uses CACHE_LINE_SIZE
```

2. Use Zero-Init Only When Needed

```
# If you'll initialize immediately, skip zero-init
v = memalign_vec(Float64, n)
fill!(v, some_value)

# If you need zeros, use memalign_clear_vec
v = memalign_clear_vec(Float64, n)
```

3. Reuse Allocations

```
# Pre-allocate buffers
buffer1 = memalign_vec(Float64, max_size)
```

```
buffer2 = memalign_vec(Float64, max_size)

# Reuse in hot loop
for data in datasets
    process!(buffer1, data)
    transform!(buffer2, buffer1)
end
```

4. Verify Type Stability

```
using Test

# Check that functions are type-stable
@inferred memalign_vec(Float64, 100)
@inferred memalign_clear_vec(Int32, 50)
@inferred alignment(v)
```

Examples Gallery

Example 1: Basic SIMD Operation

```
using LoopVectorization, AlignedAllocs

function compute!(y, x)
    @turbo for i in eachindex(y, x)
        y[i] = sqrt(abs(x[i])) + x[i]^2
    end
end

n = 10000
x = memalign_vec(Float64, n)
y = memalign_vec(Float64, n)

rand!(x)
compute!(y, x)
```

Example 2: Multi-threaded Processing

```
using Base.Threads

function parallel_process(data)
    # Each thread gets aligned buffer to avoid false sharing
    buffers = memalign_seq(Float64, nthreads(), length(data) ÷ nthreads())
```

```

@threads for i in 1:nthreads()
    chunk_start = (i-1) * length(buffers[i]) + 1
    chunk_end = i * length(buffers[i])
    copyto!(buffers[i], view(data, chunk_start:chunk_end))
    process_chunk!(buffers[i])
end

return reduce(vcat, buffers)
end

```

Example 3: FFT with Aligned Buffers

```

using FFTW, AlignedAllocs

function aligned_fft(n)
    # Allocate aligned buffers for better FFT performance
    input = memalign_vec{ComplexF64}(n)
    output = memalign_vec{ComplexF64}(n)

    # Initialize input
    for i in 1:n
        input[i] = complex(sin(2π * i / n), 0.0)
    end

    # Compute FFT
    plan = plan_fft!(input)
    mul!(output, plan, input)

    return output
end

```

Example 4: Cache-Aligned Matrix

```

function make_aligned_matrix(m, n, ::Type{T}=Float64) where T
    # Align each row to cache line
    row_stride = max(n, AlignedAllocs.CACHE_LINE_SIZE ÷ sizeof(T))
    buffer = memalign_clear_vec{T}(m * row_stride)

    # Create matrix view with proper stride
    return reshape(buffer, row_stride, m)'[1:m, 1:n]
end

# Each row starts at cache-line boundary
A = make_aligned_matrix{100, 100}

```

FAQ

Q: When should I use aligned allocations?

A: Use aligned allocations when:

- Performing SIMD operations (especially with `@turbo` or `@simd`)
- Working with multi-threaded code (to prevent false sharing)
- Interfacing with C/Fortran libraries that require alignment
- Optimizing cache-critical inner loops

Q: What's the performance overhead?

A: Minimal. The only overhead is:

- One system call at allocation (same as regular allocation)
- Slightly more memory (padding for alignment)
- Zero-initialization overhead (only for `memalign_clear_*`)

Q: Can I use this with GPU arrays?

A: No, `AlignedAllocs` is for CPU memory only. For GPU memory, use `CUDA.jl` or `AMDGPU.jl`.

Q: Will alignment improve all my code?

A: No. Alignment helps primarily:

- SIMD-vectorizable loops
- Cache-critical access patterns
- Multi-threaded code with shared cache lines

It won't help:

- Code limited by memory bandwidth
- Algorithms with poor cache locality
- Code that doesn't vectorize

Q: How do I check if alignment is working?

A: Use the `alignment` function and verify with assertions:

```
v = memalign_vec(Float64, 1000; align=256)
@assert alignment(v) >= 256
@assert alignment(v) % 256 == 0
```

Q: Can I resize aligned arrays?

A: No. `resize!` may reallocate and lose alignment. Pre-allocate to maximum size instead:


```
# Pre-allocate max size
buffer = memalign_vec(Float64, max_size)
# Use view for actual size
v = view(buffer, 1:actual_size)
```

See Also

Related Packages:

- [LoopVectorization.jl](#) - SIMD loop optimization
- [FixedSizeArrays.jl](#) - Type-stable fixed-size arrays
- [SIMD.jl](#) - Explicit SIMD operations
- [CpuId.jl](#) - CPU feature detection

External Documentation:

- [Intel® Optimization Manual](#)
 - [Julia Performance Tips](#)
 - [POSIX memalign](#)
-

Last updated: 2025-10-26